Smart Multi-Level Parking Optimization

An Online Min-Cost Assignment Approach

Project 4 - Combinatorial Optimization & Metaheuristics

Team

Houaria Djabir · Yasmine Meriche · Nacer Eddine Missouni

June 9, 2026

Contents

1 Introduction and Problem Statement

Cars arrive at a multi-level car park throughout the day, and each car must be given a parking bay *the moment it arrives*. The operator wants to serve the demand as well as possible: park as many cars as physically possible and, among the parked cars, keep the total *inconvenience* low. Inconvenience is a deliberately broad notion. It includes the walking distance from the bay to the exit, an electric vehicle (EV) that fails to get a charging bay, a long-staying staff member occupying a prime near-exit spot that a short-stay visitor needed, a small car wasting a scarce large bay, and the crowding of one floor while other levels stay empty.

The single most important characteristic of this problem, the one that shapes every design decision in the project, is that the system is **online**. We do not have the luxury of seeing the whole day at once and computing a globally optimal schedule. Instead, decisions arrive as a stream and must be committed immediately and irrevocably. The remainder of this section makes the model precise; the following sections describe the data, the shared cost function, the six solution methods, the simulation engine, the experimental results, and their interpretation.

1.1 The constraints

The model respects six constraints, listed here in the order of their consequence for the algorithm design:

Online.
At minute t we only know the cars that have *already* arrived. We never observe future arrivals.

Unknown duration.
The decision policy does not know how long a car will stay. A bay becomes free only when its car actually leaves. The simulation *environment* knows the true departure time (it must, to release bays), but this information is hidden from the decision method.

Irrevocable.
Once a car is parked it cannot be moved to another bay.

Simultaneity.
Cars that arrive in the same minute are all standing at the gate at the same instant, so they are decided *together* and matched to the bays that are free at that moment.

Size rule (hard).
A *Large* car can only use a *Large* bay. Compact and Standard cars fit any bay. This is the one inviolable feasibility constraint.

One car per bay.
A bay holds exactly one car until that car leaves; it can then be reused.

1.2 Why the problem reduces to a per-minute assignment

Because several cars frequently arrive in the same minute, and because no method may look into the future or reason about durations, the genuine decision faced at every instant is a **min-cost assignment** of *that minute's* arriving cars to the bays that are free *right now*. There is no interval reasoning, no look-ahead over future arrivals, no duration-aware packing, those would all violate the online constraint. This reduction is the conceptual backbone of the whole project: a hard online sequential decision problem is decomposed into a sequence of small, well-defined combinatorial assignment problems, one per minute, each of which the six methods solve in a different way. The only forward-looking component permitted is a lightweight *anticipation* signal built purely from the history already observed, used by one method (Section 4).

2 Data and Exploratory Analysis

2.1 The vehicle stream

The demand is a fixed set of 1000 cars stored in `data/vehicles.csv`. Each car record carries its type (Compact, Standard or Large), an EV flag, a user type (Staff or Visitor), and an arrival and departure minute-of-day. The departure is used only by the environment to release bays; the policy never sees it.

This vehicle set was derived from a public IIoT smart-parking dataset (`IIoT_Smart_Parking_Management.csv`) by the preparation notebook `data/data_prep.ipynb`. The derivation:

- maps the source vehicle weight to a size class (Compact ≤ 1200 kg, Standard ≤ 1800 kg, Large > 1800 kg);
- normalises the user type, a record is *Staff* if it was already marked Staff or if its parking duration is at least 7 hours, otherwise *Visitor*, which is what gives staff their characteristic long stays;
- synthesises arrival times that follow a realistic non-residential profile: 10 % of arrivals in 07:00–11:00, 60 % in 11:00–17:00, and 30 % in 17:00–23:00, with 1–5 minute inter-arrival spacing inside each band, and longer stays scheduled earlier so that all departures fall before 23:00.

2.2 The ten car parks

Ten candidate car-park layouts are stored as `data/parking1.json` ... `parking10.json`. They range from 98 to 486 bays across 1 to 4 floors. Each bay declares its floor, size (Small or Large), a charger flag, and a distance to the exit. At load time we additionally tag the bottom 25 % nearest bays as "near exit". The loaders build the two core data structures used everywhere in the code:

```
Car   id, v_type, is_ev, user_type, arrival, departure
Bay   id, floor, size, has_charger, near_exit, distance_to_exit
```

2.3 Demand profile and the capacity ceiling

Figure 1 shows the four faces of the demand. Arrivals peak in the midday band; durations are heavily right-skewed (most cars stay a short while, a minority stay all day); the fleet is dominated by Compact and Standard cars with a Large minority; and EVs and Staff are both minorities (EVs around 20 %, Staff only 51 of the 1000 cars).

A single number governs how many cars can ever be parked: the *peak concurrent demand*, obtained by sweeping the arrival/departure events and tracking how many cars are simultaneously present. That peak is **355 cars at once**. Figure 2 compares each park's capacity against this line. Three parks (3, 8 and 10) have fewer than 355 bays, so they physically cannot hold everyone at the peak, no matter how clever the policy. This single fact explains why every method will later park almost exactly the same fraction of cars: capacity sets the headline number, and the method only sets the *quality* of the placements.

3 The Shared Cost Function

Every method minimises the *same* cost when it matches a car to a bay. Centralising the objective in one function is what makes the six methods comparable: they differ only in *how* they search for the minimum, never in *what* they consider good. The cost is deliberately *scarcity-aware*, it does not merely look at the car in front of us, it also protects the bays that are scarce and valuable.

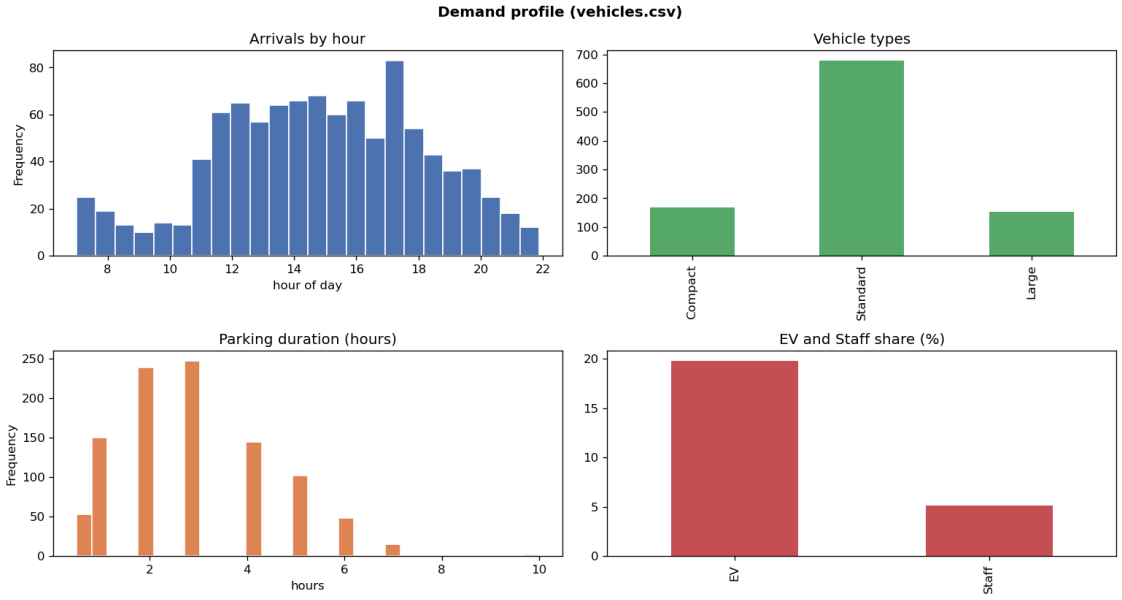


Figure 1: Demand profile of the 1000-car stream: arrivals by hour, vehicle-type mix, parking-duration distribution, and the EV / Staff shares.

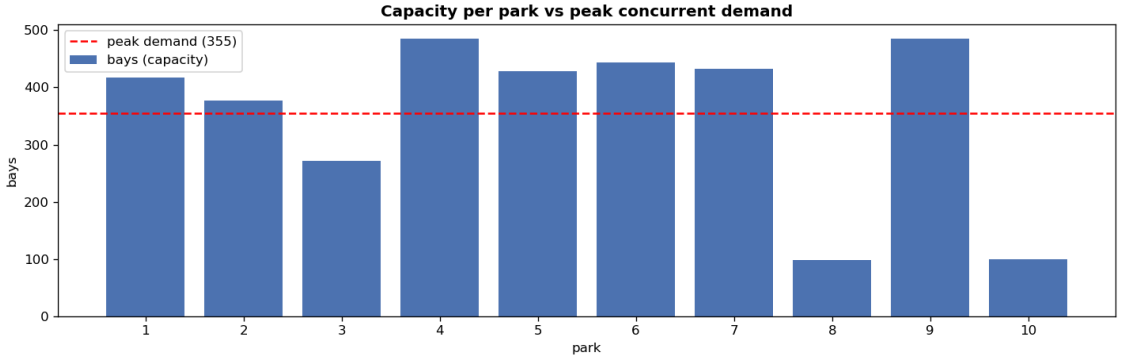


Figure 2: Capacity (bays) per park versus the peak concurrent demand of 355 cars. Parks below the dashed line saturate at peak.

For a car c placed in bay b, let d be the bay's distance to the exit and $D_{\max}$ the farthest distance in the park. The cost is

$$\begin{aligned}\text{cost}(c,b) \;=\; & \underbrace{\Big[\mathbf{1}[c \text{ is Staff}]\,(D_{\max}-d) + \mathbf{1}[c \text{ is Visitor}]\,d\Big]}_{\text{access term (staff pushed to the back)}} \;+\; \underbrace{w_{\text{fl}}\,(\text{floor}(b)-1)}_{\text{upper-floor penalty}} \\ & +\; \underbrace{p_{\text{ev}}\,\mathbf{1}[c \text{ is EV} \wedge b \text{ has no charger}]}_{\text{EV without a charger}} \;+\; \underbrace{c_w\,\mathbf{1}[b \text{ has a charger} \wedge c \text{ not EV}]}_{\text{charger preservation}} \\ & +\; \underbrace{s_w\,\mathbf{1}[b \text{ is Large} \wedge c \text{ not Large}]}_{\text{large-bay preservation}} \;+\; \underbrace{f_{\text{ld}} \cdot \rho_{\text{floor}(b)}}_{\text{floor load balancing}}\end{aligned} \tag{1}$$

where $\rho_{\text{floor}(b)}$ is the current occupancy fraction of the chosen floor. Each term encodes one piece of the inconvenience notion from Section 1:

Walking distance.
The base inconvenience for a visitor is simply d.

The staff flip.
Staff stay long, so for a staff car the access cost is $D_{\max} - d$ instead of d. This deliberately pushes long-stay staff to the *back* of the park, freeing the prime near-exit bays for the many short-stay visitors who churn through them. Visitors keep the natural d, so they prefer near bays.

Floor penalty.
Upper floors cost w_{fl} per level above the ground, since reaching them takes longer.

EV needs a charger.
A large penalty p_{ev} if an electric car is placed without a charger.

Charger preservation.
A penalty c_w if a non-EV takes a charger bay, so that scarce chargers stay free for EVs.

Large-bay preservation.
A penalty s_w if a non-large car takes a large bay, so that scarce large bays stay free for large cars.

Floor load balancing.
A mild penalty proportional to how full the chosen floor already is, which spreads cars across levels.

All weights are tunable constants declared at the top of the notebook (Table 1). The two *preservation* terms (c_w and s_w) are the only ones the anticipation module rescales, and only for one method.

Table 1: Tunable cost weights (units: metres-equivalent; raw distance is ∼5–60 m).

Symbol	Constant	Value
p_{ev}	`EV_PEN`	80.0
c_w	`CHARGER_WASTE`	25.0
s_w	`SIZE_WASTE`	25.0
w_{fl}	`W_FLOOR`	15.0
f_{ld}	`FLOOR_LOAD`	20.0
λ	`LAMBDA` (anticipation)	3.0
–	`UNASSIGNED`	10^6

Leaving a car unparked is modelled by a sentinel cost `UNASSIGNED` $= 10^6$, which dwarfs any real placement cost. The optimiser therefore parks a car whenever a feasible bay exists, and only ever "chooses" to leave a car on the street when the park is genuinely full. This single trick lets the same min-cost machinery handle both the quality objective and the capacity constraint at once.

4 The Six Solution Methods

Every minute we must match the cars that just arrived to the bays free right now, at minimum total cost. That is a min-cost assignment problem, and the six methods are six different ways to solve it. The first is a naive baseline; the next three are exact; the last two are metaheuristics.

4.1 Shared scaffolding

Two helpers are shared by all the joint solvers. The `compatible` test enforces the single hard size rule. The `_pool` helper restricts the search to a small candidate set: for a group of g cars it

gathers, for each car, its $g + 5$ cheapest compatible free bays and takes the union. The optimal matching of g cars never needs more bays than this, so the candidate pool keeps every solver's search tiny without sacrificing optimality. Single-car minutes (the common case) bypass the machinery entirely and just take the cheapest compatible bay.

4.2 Baseline, greedy nearest bay

Each car, one at a time, takes the nearest free compatible bay, using distance *only*. There is no joint matching and none of the scarcity terms. This is the obvious, naive approach and the yardstick everything else is measured against.

4.3 ILP, exact integer program

The matching is written as a binary program with variables $x_{c,b} \in \{0, 1\}$ and solved exactly by CBC through `pulp`:

$$\min \quad \sum_{c,b} \text{cost}(c, b)\, x_{c,b} + \sum_{c} \text{UNASSIGNED}\Big(1 - \sum_{b} x_{c,b}\Big) \tag{2}$$

$$\text{s.t.} \quad \sum_{b} x_{c,b} \le 1 \quad \forall c \qquad \text{(each car at most one bay)} \tag{3}$$

$$\sum_{c} x_{c,b} \le 1 \quad \forall b \qquad \text{(each bay at most one car)} \tag{4}$$

This is the formal, declarative statement of the problem and the exact reference the others are checked against. `pulp` is an optional dependency: if it is absent every ILP step is skipped and Branch & Bound remains as the exact reference.

4.4 B&B, branch and bound

A depth-first search over the cars that assigns each car to a bay or leaves it unparked, pruning any branch whose accumulated cost plus an admissible suffix lower bound already meets or exceeds the best solution found so far. It is warm-started with a greedy matching and bounded by a time limit (2 s). It reaches the same optimum as ILP, by explicit search rather than by a solver.

4.5 SA, simulated annealing

A metaheuristic local search that starts from a greedy matching and improves it with small moves, reassign one car to a different free bay, or swap two cars' bays, accepting worse moves with the usual Metropolis probability $\exp(-\Delta/T)$ to escape local minima. The temperature cools geometrically and the iteration budget scales with the minute size ($\min(1500, 40g)$). It approximates the optimum.

4.6 GA, genetic algorithm

A metaheuristic that evolves a population of matchings. Each chromosome maps cars to bays; tournament selection, one-point crossover and swap mutation produce offspring, and a repair step guarantees that no two cars share a bay. Elitism preserves the two best individuals each generation. Population and generation counts scale with the minute size. It too approximates the optimum.

4.7 Hungarian, Kuhn–Munkres with anticipation

The Kuhn–Munkres algorithm solves the assignment problem exactly in $O(n^3)$. The cost matrix has one row per car and one column per candidate bay, plus g extra "street" columns of cost

`UNASSIGNED` that let a car stay unparked; incompatible cells are set above `UNASSIGNED` so they are never chosen. Crucially, Hungarian is the only method carrying an **always-on anticipation** signal. Using the arrivals seen so far and the current scarcity of chargers and large bays, it scales the two preservation weights

$$c_w = \text{CHARGER\_WASTE}\,(1 + \lambda\, p_{\text{ev}}\, \rho_{\text{ch}}), \qquad s_w = \text{SIZE\_WASTE}\,(1 + \lambda\, p_{\text{lg}}\, \rho_{\text{lg}}), \tag{5}$$

where $p_{\text{ev}}, p_{\text{lg}}$ are the observed EV and large-car fractions and $\rho_{\text{ch}}, \rho_{\text{lg}}$ the current occupancy of charger and large bays. When chargers are filling up and the history says EVs keep coming, the penalty for letting a non-EV waste a charger rises, so Hungarian reserves those scarce bays for the high-need cars still to arrive. This look-ahead uses only past observations, it never peeks at future arrivals, so it honours the online constraint.

Algorithm 1 One minute, the Hungarian method

1: **Input:** arriving group, free bays, $D_{\max}$, floor occupancy, weights c_w, s_w
2: **if** group has one car **then return** its cheapest compatible bay
3: **end if**
4: pool ← union of each car's g+5 cheapest compatible bays
5: build cost matrix M (cars × [pool ∪ street columns])
6: assign ← KuhnMunkres(M)
7: **return** for each car, its assigned pool bay, or **None** if a street column was chosen

A note on fair comparison. ILP, B&B and Hungarian are three *exact* methods, so on the same cost they return the same optimal matching; SA and GA approximate it. But because Hungarian also applies anticipation, it minimises a slightly different, history-adjusted objective. To compare fairly, every method's *final placement* is re-scored on one common objective, the static scarcity-aware cost with no anticipation, and it is this `ref_cost` that the results report alongside the real-world KPIs.

5 The Online Simulation Engine

The engine plays each car park forward minute by minute, strictly online. For each minute t that has arrivals it performs four steps:

1. **Release.** Free the bays whose car has already departed (departure $\leq t$). This is the *only* use of the hidden real departure time, and it belongs to the environment, not the method.
2. **Observe.** Take the cars arriving this minute and the bays free now, and compute the current floor-occupancy fractions.
3. **Anticipate (Hungarian only).** Update the history counters from the arrivals just seen and rescale the two preservation weights. Every other method uses the static weights.
4. **Decide and commit.** Match the cars to bays with the chosen method, commit the choice irrevocably (book each parked car's true departure so its bay frees at the right time), and log the outcome.

For every parked car the log records whether it got a charger, the bay's distance and floor, whether a large bay was used by a non-large car, and the `ref_cost`: the chosen bay scored on the static objective, so totals are comparable across methods. From the per-car log we compute nine KPIs per run, parked percentage, total and average cost, EV-charged percentage, charger waste, large-bay waste, mean visitor and staff distance, and floor spread. All six methods are run on all ten parks; random seeds are fixed, so the experiment is fully reproducible.

6 Experimental Results

Table 2 reports each KPI averaged over the ten parks. Figures 3–5 visualise the same data.

Table 2: Mean KPIs across the ten car parks (lower is better for cost, waste and distance; higher is better for parked % and EV-charged %).

Method	Parked %	Total cost	EV chgd %	Chg waste %	Lg waste %	Vis. dist	Staff dist	Runtime s
Baseline	88.20	71 500	24.24	24.59	24.53	31.89	31.21	0.08
ILP	88.22	55 662	55.31	8.80	12.94	31.73	84.37	6.44
B&B	88.21	56 044	54.64	9.11	13.30	31.83	84.41	1.58
SA	88.24	56 076	54.69	9.11	13.27	31.85	84.36	0.31
GA	88.26	56 393	54.50	9.54	13.71	31.81	84.30	0.96
Hungarian	88.22	**55 215**	**58.40**	**7.15**	**12.39**	32.18	83.95	0.17

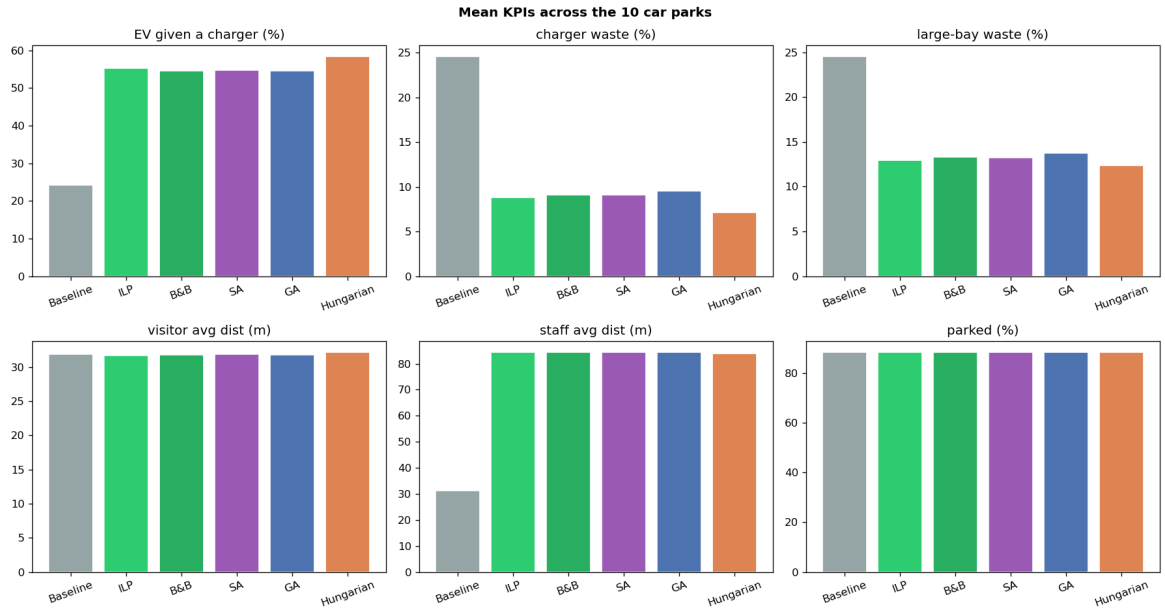


Figure 3: Mean KPIs across the ten car parks. The naive baseline (grey) is worst on every quality metric; Hungarian (orange) leads.

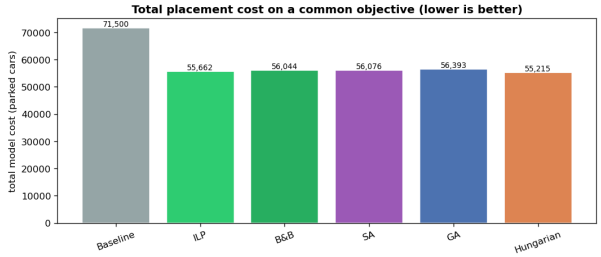
7 Interpretation

7.1 Capacity sets how many park; the method sets how well

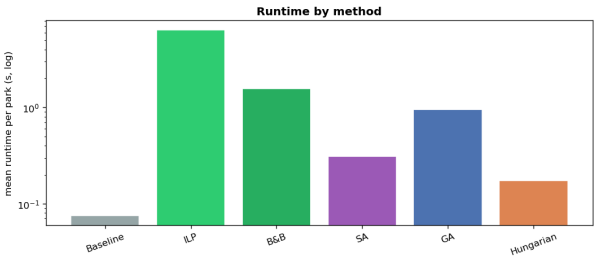
Peak concurrent demand is 355 cars, and three parks cannot hold that. Consequently every method parks about the same share of cars (around 88 %), and the parked-percentage bars are nearly identical. The interesting differences are not in *how many* cars park but in *how good* their bays are.

7.2 Baseline versus the five optimisers

The naive baseline is far worse on every quality metric. It charges only 24 % of EVs against ∼55 % for the optimisers; it wastes chargers on non-EVs 25 % of the time against ∼9 %; it wastes large bays 25 % against ∼13 %; and it parks staff at 31 m against ∼84 m. That last figure is the staff flip working: the optimisers push long-stay staff to the back, the baseline does not. This



(a) Total placement cost on the common static objective.



(b) Mean runtime per park (log scale).

Figure 4: Cost and runtime. Hungarian achieves the lowest total cost at nearly the speed of the baseline; ILP is by far the slowest.

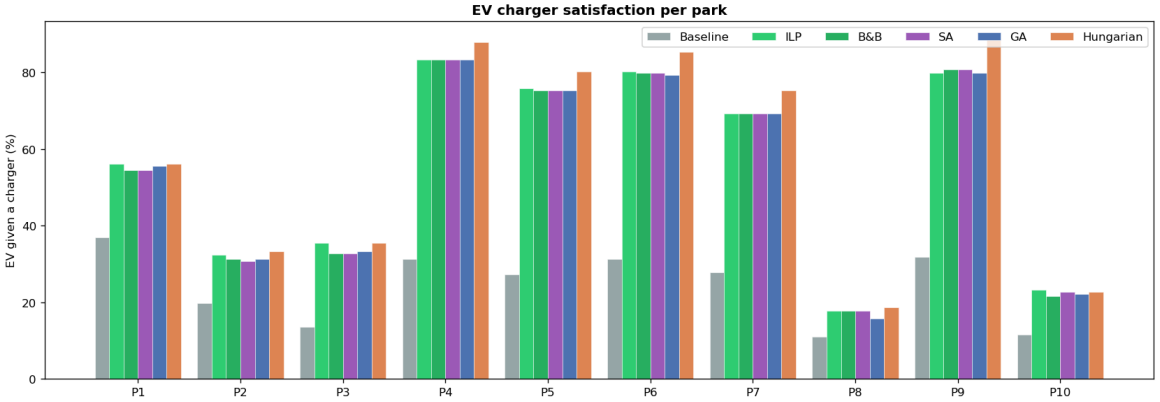


Figure 5: EV charger satisfaction per park, all six methods. Hungarian leads in almost every park; the gap widens where chargers are under pressure.

whole gap is the value of the scarcity-aware cost together with joint matching. Notice that visitor distance barely moves (∼32 m either way): staff are only 51 of 1000 cars, so freeing their bays helps the 949 visitors only slightly *on average*. The visitor win is qualitative, they keep the prime bays, not a large change in the mean.

7.3 The exact methods and the metaheuristics tie

ILP, B&B, SA and GA all land within about one point of each other. This is expected: most minutes contain only a few cars, so all four reach the same small per-minute optimum on the plain cost. ILP is the exact reference; B&B trails it only because it hits its time limit on the single 30-car opening minute and returns its warm start there; SA and GA occasionally miss the exact optimum by a hair. In effect they validate one another. The practical lesson is that *at this per-minute scale the choice between an exact method and a good metaheuristic barely matters*, the cost function dominates the outcome, not the search technique.

7.4 Hungarian plus anticipation wins, and why

Hungarian is best on every quality KPI: EV satisfaction 58 % (about 3 points above the exact ILP), the lowest charger waste (7 %) and the lowest large-bay waste (12 %). The most instructive comparison is with ILP. ILP is provably optimal *every single minute*, yet Hungarian ends the day with a *lower total cost on the very same static objective*. How can a per-minute-optimal method be beaten on the cumulative objective it optimises?

The answer is that the minutes are *not independent*: a bay given away now is gone later. ILP minimises only the current minute, so it will hand a charger bay to a non-EV whenever that is

locally cheapest, and then pay the large EV penalty later when an electric car arrives to find no charger free. Hungarian's anticipation holds those scarce bays back, so fewer penalties are paid across the whole day, which shows up directly in its lower charger and large-bay waste. In other words, a per-minute-optimal policy is still *myopic* in an online setting, and anticipation reduces that myopia. The important consequence is that anticipation is not trading cost for satisfaction here: it improves both at once.

7.5 Runtime

Runtimes per park (means) are 0.08 s (Baseline), 0.17 s (Hungarian), 0.31 s (SA), 0.96 s (GA), 1.58 s (B&B) and 6.44 s (ILP). ILP is slowest because it spins up the CBC solver on every multi-car minute. Hungarian delivers the best quality and the lowest cost at almost the speed of the baseline, because it is a specialised polynomial algorithm. This is the practical case for it: best results, near-instant.

7.6 The weakest lever: floor balancing

The floor load-balancing term is the least effective component. The distance and charger pulls toward the lower, nearer floors are far stronger than a weight of 20, so the optimisers still concentrate cars on the ground and charger floors, and on single-floor parks the term does nothing at all. It is a tunable constant: raising `FLOOR_LOAD` would trade walking distance for a more even spread across levels if that ever became the priority.

8 Conclusion

Chargers are physically scarce (15 to 77 per park for roughly 198 EVs in the stream), so no method can charge them all, the ceiling is structural, just as capacity caps the parked percentage. Within those ceilings, the project shows three clear results. First, scarcity-aware joint matching roughly *doubles* EV satisfaction over the naive baseline and *halves* the resource waste, which is the value of the cost function in Equation (1). Second, at this per-minute scale the difference between exact solvers and good metaheuristics is negligible, so a fast specialised method is preferable to a heavy general one. Third, and most interesting, a history-only anticipation signal lets the Hungarian method beat the per-minute-optimal ILP on ILP's own cumulative objective, because it counteracts the myopia inherent in online decision-making. The recommended configuration is therefore the Hungarian method with anticipation: it gives the best KPIs and the lowest total cost at near-baseline speed, while fully respecting the online, unknown-duration, irrevocable nature of the real problem.

Reproducibility. All results are produced by `main.ipynb` run top to bottom with fixed seeds; figures are written to `viz/` and the full per-park KPI table to `viz/mt_results.csv`.